

Structural Geology with apsg

Block 1 — Modern Workflow in Geology

Ondrej Lexa

Institute of Petrology and Structural Geology (IPSG)
Charles University, Prague, Czechia

8 October 2026

Motivation

Objective

Manipulate, analyse and project orientation data — the things structural geologists need every day — in a few lines of reproducible code.

1. Basic objects: `vec`, `lin`, `fol`, `pair`, `fault`
2. Feature sets and field data from CSV
3. Stereonets: points, great circles, density contours
4. Population analysis: mean, Fisher statistics, eigenvectors, fold axes
5. Saving vector graphics for publication

Notebook: `notebooks/01_apsg_basics.ipynb`

Why Objects?

Two numbers in a spreadsheet

120, 60 — a lineation? a plane? dip direction or strike? Every calculation (poles, intersections, rotations, means) has to re-encode that meaning by hand.

An apsg object

`fol(120, 60)` *is* a plane. It knows its pole, how to intersect with another plane, how to rotate, how to plot itself, and what its mean with other planes means.

apsg in one sentence

A Python package for structural geologists: orientation **features**, **sets** of features, **tensors**, and **plots** that understand them.

Basic Objects

```
from apsg import *  
  
u = vec(1, -2, 3) # components  
v = vec(120, 60) # azimuth, inclination  
  
abs(u) # length  
u + v # sum  
u @ v # dot product  
u.cross(v) # cross product  
u.angle(v) # angle in degrees  
u.rotate(v, 45) # rotate u around v
```

- Geographic frame: x = north, y = east, z = down
- Ordinary vector algebra with Python operators
- The foundation for every other feature type

Lineations and Foliations

```
l = lin(120, 60) # trend / plunge
f = fol(216, 62) # dip direction / dip

fol("N30E,40NW") # quadrant notation
with apsg_conf_context(notation="rhr"):
    print(f) # S:126/62 (strike RHR)

lin(f) # pole to the plane
fol(l) # plane normal to line
lin(f.dipvec()) # dip vector
lin(f.rake(30)) # line with rake 30
```

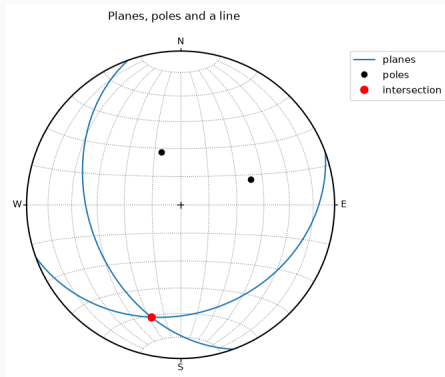
- `lin` and `fol` are **axial** and always lower hemisphere
- A foliation is stored by its **pole**
- Input and output notation is configurable (`dd`, `rhr`, `quadrant`)
- Conversions are explicit:
`lin(f)`, `fol(l)`

The Geological Cross Product

```
f1, f2 = fol(250, 40), fol(160, 30)

f1 ** f2 # intersection: L:195/25
lin(250, 40) ** lin(160, 30)
           # plane through both lines
f1.angle(f2) # 48.4 deg

s = StereoNet()
s.great_circle(f1, f2, label="planes")
s.pole(f1, f2, color="k", label="poles")
s.line(f1 ** f2, color="r", ms=8)
s.show()
```



Pairs and Faults

```
p = pair(120, 40, 162, 28) # plane + line in it (e.g. S + stretching L)
p.misfit # 3.56 deg: measured line not exactly in plane
p.fol, p.lin, p.rake # adjusted, perfectly orthogonal pair

ft = fault(120, 60, 110, 58, "n") # pair + sense of movement
ft.p, ft.t, ft.m # P axis, T axis, movement plane
fault(140, 30, 110, 26, "r") # sense: 1/"n", -1/"r", "s", "d"
fault(fol(120, 80), lin(32, 10), "s")
```

- Real measurements never fit perfectly — pair corrects them symmetrically and reports the misfit
- fault is the starting point for kinematic and stress analysis (Block 2)

Feature Sets & Stereonets

Feature Sets

```
L = G([lin(120, 60), lin(116, 50), lin(132, 45), lin(90, 60)]) # -> LineationSet
L[0], len(L), L[1:3] # behaves like a list

cluster = linset.random_fisher(position=lin(120, 40), kappa=30, n=100)
S = folset.from_csv("planes.csv", acol="azi", icol="inc")
```

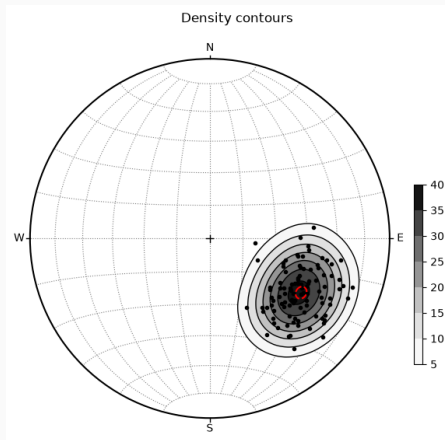
Single feature	Set
vec, lin, fol	vecset, linset, folset
pair, fault	pairset, faultset
any list of features	G([...]) picks the right set type

Sets add group methods: mean, statistics, orientation tensor, rotation, ...

Stereonet

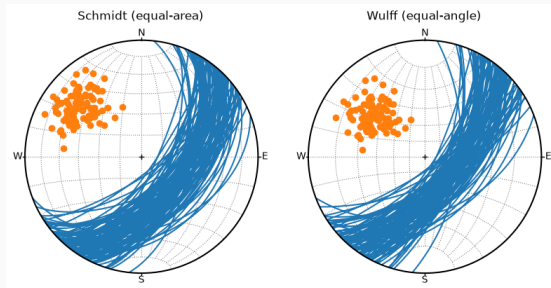
```
s = StereoNet(title="Density contours")
s.contour(cluster, levels=8, colorbar=True)
s.line(cluster, color="k", ms=3)
s.confidence(cluster, method="fisher",
             level=0.95, color="r")
s.show()
```

s.line / s.pole	points
s.great_circle	planes
s.contour	density (Kamb)
s.confidence	cone around mean
s.pair / s.fault	composite features



Projection Options

- `StereoNet()` — equal-area (Schmidt), lower hemisphere
- `kind="equal-angle"` — Wulff net
- `hemisphere="upper"`
- `grid=False, title=...`
- Global look via `apsg_conf`, e.g. `apsg_conf.figsize`



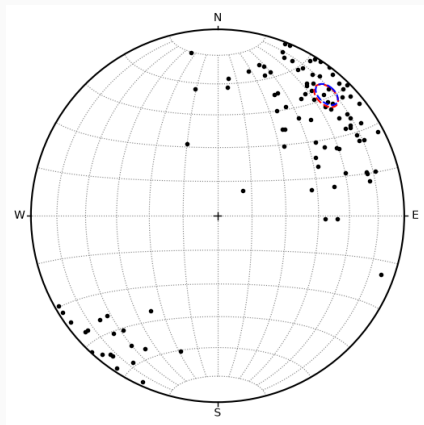
Schmidt vs. Wulff net (apsg docs)

Population Analysis

Mean Orientation and Fisher Statistics

```
cluster.R() # mean: L:121/41  
st = cluster.fisher_statistics()  
st["k"], st["alpha"] # 31.0, 2.6 deg  
cluster.var() # spherical variance
```

- $R()$ — resultant (mean) direction
- k — precision; α — 95 % cone
- Also `bingham_statistics`,
`watson_statistics`, `bootstrap`



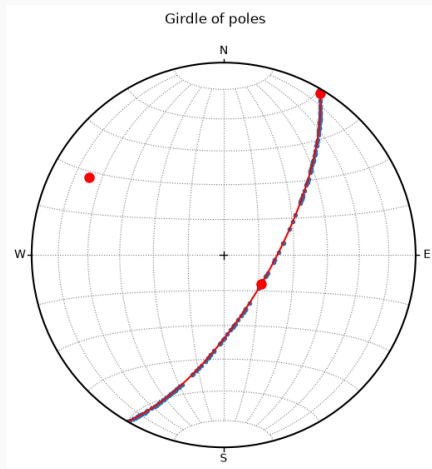
Fisher vs. bootstrap cones (apsg docs)

Orientation Tensor and Eigenvectors

```
ot = girdle.ortensor()
ot.eigenvalues() # [0.625 0.375 0. ]
ot.eigenlins() # principal axes
ot.eigenfols(2) # best-fit girdle plane
ot.P, ot.G, ot.R # point/girdle/random

s = StereoNet()
s.pole(girdle, ms=3)
s.line(*ot.eigenlins(), color="r", ms=8)
s.great_circle(ot.eigenfols(2), color="r")
```

Works for clusters *and* girdles — where a simple mean fails.

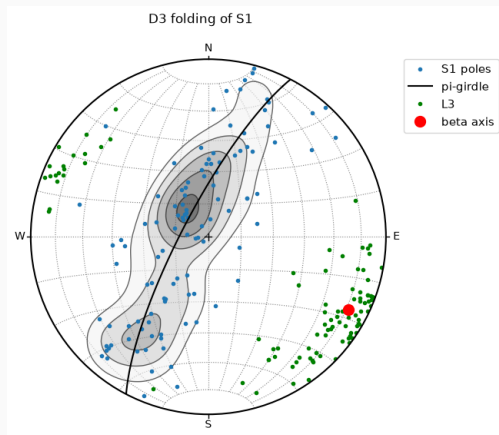


Fold Axis from Folded Foliations

```
from apsg.pandas import pd
df = pd.read_csv("data/structures.csv")
S1 = df[df.structure == "S1"].fol(name="S1")
L3 = df[df.structure == "L3"].lin(name="L3")

beta = S1.ortensor().eigenlins(2)
# beta (S1 girdle axis): L:118/12
# mean L3 : L:122/8
```

Poles of a cylindrically folded surface lie on a π -girdle; the **smallest eigenvector** is the fold axis β . Here it matches the independently measured L3 within 6°.



Field Data & Output

Reading Field Data with pandas

```
from apsg.pandas import pd, gbf # registers df.lin, df.fol, df.vec, df.fault

df = pd.read_csv("data/structures.csv") # columns: site, structure, azi, inc
S1 = df[df.structure == "S1"].fol(name="S1") # -> FoliationSet
df[df.structure == "L3"].lin.groupby("site").apply(gbf.mean) # mean L3 per site
df[df.structure == "S1"].fol.plot(kind="pole") # quick stereonet

wide = pd.read_csv("data/structures_wide.csv") # S1_azi, S1_inc, L1_azi, ...
wide.fol.set_columns(azi="S1_azi", inc="S1_inc")
S1_wide = wide.fol(name="S1") # rows with missing values skipped
```

Excel works the same: `pd.read_excel("field.xlsx", sheet_name="S1")`

Saving Vector Graphics

```
s = StereoNet(title="S1 and L3")
s.great_circle(S1, color="0.7", lw=0.5)
s.line(L3, color="g", ms=4, label="L3")
s.line(beta, color="r", ms=10, label="beta")
s.savefig("output/S1_L3.pdf") # vector - edit in Inkscape/Illustrator
s.savefig("output/S1_L3.svg") # vector
s.savefig("output/S1_L3.png", dpi=300) # raster
```

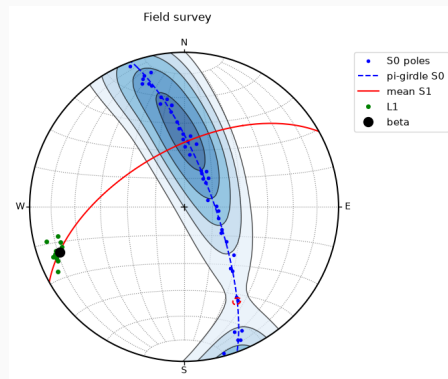
- The file extension selects the format; all other keywords go to matplotlib

Hands-on

Hands-on Exercise

`data/field_measurements.csv` — bedding S0, cleavage S1 and intersection lineation L1 from 12 stations

1. Load it; build sets for S0, S1, L1
2. S0 poles with contours, S1 as great circles, L1 as points
3. Fold axis β from S0; compare with mean L1 — does β lie in the mean S1 (axial-planar cleavage)?
4. Mean S1 and its Fisher α_{95}
5. Save the stereonet as PDF and SVG



Target result (solution in the notebook)

Recap

Recap

Task	Code
Create features	<code>vec(1,2,3), lin(120,60), fol(216,62), pair, fault</code>
Convert	<code>lin(f) (pole), fol(l), f1 ** f2 (intersection)</code>
Collections	<code>G(...), linset, folset, df.fol(), df.lin()</code>
Plot	<code>StereoNet() → line/pole/great_circle/contour → savefig</code>
Statistics	<code>.R(), .fisher_statistics(), .ortensor().eigenlins()</code>
Fold axis	<code>S.ortensor().eigenlins(2)</code>

Resources: github.com/ondrolexa/apsg — docs with tutorial notebooks

Next: faults, tensors and stress inversion